

# Coding Patterns Interview Field Guide

Recognition cues, invariants, proof ideas, JavaScript templates, and a disciplined solve loop

**WHAT THIS IS FOR**

Timed algorithm rounds. Seniority does not change the algorithm, but it changes how clearly you scope, prove, test, and recover when an approach is wrong.

**INTERVIEW RULE**

A cheatsheet should reduce recall time. It should not replace explaining the invariant, tradeoff, and failure mode in your own words.

**HOW TO USE IT**

- Read pages 2–3 before a practice block.
- Middle pages: only after identifying the category.
- Final checklist: during timed mocks.
- Re-code the templates from memory; passive rereading is not preparation.

| \$ | FOCUS                                    | USE IT FOR                                   |
|----|------------------------------------------|----------------------------------------------|
| 02 | Solve loop + priorities                  | communication, invariant, tests, study order |
| 03 | Hash + prefix                            | lookup, frequency, subarray counts           |
| 04 | Pointers + windows                       | sorted discard, contiguous ranges            |
| 05 | Stacks + binary search                   | waiting answers, monotonic predicates        |
| 06 | Linked structures                        | reversal, cycles, dummy nodes                |
| 07 | Trees + tries                            | information flow, recursion, BFS             |
| 08 | Heaps + selection + <b>HEAP TEMPLATE</b> | top-k, merge, running median                 |
| 09 | Graphs + <b>CYCLE DETECTION</b>          | DFS/BFS, topo, union-find                    |
| 10 | Weighted graphs                          | Dijkstra, MST, Bellman-Ford                  |
| 11 | Backtracking                             | decision trees, pruning, dedupe              |
| 12 | Dynamic programming                      | state, transition, bases, order              |
| 13 | Greedy + intervals                       | proof, scheduling, merging                   |
| 14 | Final decision tree                      | debugging, edge cases, rehearsal             |

## 02 The solve loop and pattern priorities

### A. THE 35-45 MINUTE CODING LOOP

- **0-4:** restate input/output, examples, constraints, mutation, duplicates, ordering, empty/invalid behavior.
- **4-8:** name brute force and its bottleneck; propose pattern; state why input structure allows it.
- **8-12:** define invariant/data structure and complexity **before** code.
- **12-28:** code in coherent chunks while narrating. Prefer boring control flow.
- **28-35:** dry-run normal + edge case; fix from the invariant, not random edits.
- **35+:** optimize or discuss alternatives only after a correct solution.

### B. THE FOUR SENTENCES

- **Brute force:** "The direct solution is  $O(\dots)$  because ..."
- **Observation:** "The sorted/contiguous/monotonic structure lets me ..."
- **Invariant:** "At loop start, this map/window/stack represents ..."
- **Complexity:** "Each element enters/leaves once, so  $O(n)$ , with  $O(k)$  state."

### C. PRIORITY TIERS

| TIER               | PATTERNS                                                                      |
|--------------------|-------------------------------------------------------------------------------|
| 1 — automatic      | hash, two pointers, sliding window, prefix sum, stack, binary search, BFS/DFS |
| 2 — fluent         | linked list, trees, heap, intervals, topo, backtracking, 1-D DP               |
| 3 — targeted       | union-find, Dijkstra, 2-D DP, trie, monotonic deque, greedy proof             |
| 4 — role-dependent | MST, Bellman-Ford, bit tricks, advanced geometry, segment/Fenwick             |

### D. RECOGNITION MAP

| CUE                         | REACH FOR               |
|-----------------------------|-------------------------|
| pair / frequency / seen     | hash map/set            |
| sorted pair or palindrome   | two pointers            |
| contiguous longest/shortest | sliding window          |
| subarray sum / range query  | prefix sum              |
| next greater / nesting      | stack / monotonic stack |
| monotonic yes/no answer     | binary search           |
| k best / cheapest frontier  | heap                    |
| connectivity / grid         | DFS/BFS/union-find      |
| prerequisites               | topological sort        |
| all combinations            | backtracking            |
| overlapping subproblems     | DP                      |
| ranges                      | sort intervals          |

### E. RECOVERY WHEN STUCK

- Shrink the example; write brute force; identify repeated work or ordering you are ignoring.
- Ask which dimension can be made **monotonic**: index, window validity, answer feasibility, stack order, visited state.
- If optimization stalls, implement correct brute force and state the missing insight. Working code beats silent searching.

#### STAFF+ SIGNAL

Do not over-engineer the coding round. The signal is crisp contracts, a justified invariant, clean code, tests, and calm recovery — not architecture commentary during every loop.

## 03 Hash maps, sets, and prefix sums

### A. HASH RECOGNITION

- Need  $O(1)$  expected membership, complement, frequency, canonical grouping, dedupe, or last-seen position.
- **Map when the value matters; Set for membership.** State  $O(n)$  expected time and  $O(k)$  distinct-state space.

### B. COMPLEMENT / FREQUENCY

```
function twoSum(a, target) {
  const seen = new Map();
  for (let i = 0; i < a.length; i++) {
    const need = target - a[i];
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(a[i], i);
  }
  return null;
}

const freq = a.reduce(
  (m, x) => m.set(x, (m.get(x) ?? 0) + 1), new Map());
```

- Order of **check vs insert** decides whether the same element can match itself. Test duplicate values.

### C. CANONICAL KEYS

- Group equivalent items by normalized key: sorted characters  $O(L \log L)$  or fixed alphabet counts  $O(L)$ .
- Compound JS keys need a stable representation or nested Maps; array/object literals compare by **identity**.
- Beware delimiter collisions when joining values; length-prefix or nested structure if the domain is unconstrained.

### D. PREFIX SUM

- Prefix  $p[i]$  = sum before index  $i$ . Range  $[l, r)$  sum =  $p[r] - p[l]$ . Build  $O(n)$ , query  $O(1)$ .

- Subarray sum  $K$ : at the current prefix sum, need an earlier prefix equal to  $\text{sum} - K$ . Store counts, seeded with  $0 \rightarrow 1$ .

```
function subarraySum(a, k) {
  const count = new Map([[0, 1]]);
  let sum = 0, ans = 0;
  for (const x of a) {
    sum += x;
    ans += count.get(sum - k) ?? 0;
    count.set(sum, (count.get(sum) ?? 0) + 1);
  }
  return ans;
}
```

- **Why seed zero?** It counts a valid subarray beginning at index 0.

### E. PREFIX VARIANTS

- Equal 0/1 counts: map 0 to  $-1$  contribution and seek prefix difference 0.
- Divisible by  $K$ : equal prefix remainders; **normalize negative remainders in JS** — `%` preserves the dividend's sign: `((x % k) + k) % k`.
- 2-D range sum: prefix rectangle and inclusion-exclusion.

### F. BUG CHECK

- Missing vs stored zero: use `has/??`, not truthiness.
- Return count vs indices vs values; duplicate policy; empty answer.
- Input magnitude may exceed `Number.MAX_SAFE_INTEGER`; ask constraints.

#### INVARIANT

Before processing  $a[i]$ , the map summarizes exactly the earlier elements/prefixes. Never add the current item too early unless the problem allows self-pairing.

## 04 Two pointers and sliding windows

### A. TWO POINTERS

- **Converging** on sorted data: the comparison tells which side cannot participate, so discard it.
- **Reader/writer**: scan with fast; write kept/compacted values at slow.
- **Same direction with gap**: kth from end, cycle/middle variants, merge streams.

```
let l = 0, r = a.length - 1;
while (l < r) {
  const sum = a[l] + a[r];
  if (sum === target) return [l, r];
  if (sum < target) l++; else r--;
}
```

### B. 3SUM / DUPLICATES

- Sort; fix i; run two-pointer on the suffix. Skip equal i values, and equal l/r values **after recording**.
- Complexity  $O(n^2)$  after  $O(n \log n)$  sort. Clarify whether mutating input is allowed.

### C. VARIABLE WINDOW

```
let l = 0, best = 0;
for (let r = 0; r < a.length; r++) {
  add(a[r]);
  while (!valid()) remove(a[l++]);
  best = Math.max(best, r - l + 1);
}
```

- The window is **contiguous**. State must make add/remove/valid  $O(1)$ : count map, sum, distinct count, max frequency.
- **Longest valid**: shrink while invalid, record after.  
**Shortest valid**: shrink while valid, recording before removal.

### D. FIXED WINDOW / MONOTONIC DEQUE

- Fixed k: add right, remove index  $r-k$ , record when size is k.

- Sliding max: deque stores indices with decreasing values; pop expired front and dominated back. Each index enters/leaves once  $\rightarrow O(n)$ .

```
const dq = []; // indices, values decreasing
for (let r = 0; r < a.length; r++) {
  while (dq.length && a[dq.at(-1)] <= a[r]) dq.pop();
  dq.push(r);
  if (dq[0] <= r - k) dq.shift(); // or head index
  if (r >= k - 1) out.push(a[dq[0]]);
}
```

### E. WHEN THE WINDOW FAILS

- If negative values make sum validity non-monotonic, shrinking no longer guarantees progress toward validity. Use **prefix sum + map** or another structure.
- If the target is a **subsequence** (not contiguous), a window is usually wrong. Consider DP, greedy, or pointers over sorted data.

### F. CLASSIC LAST-SEEN JUMP

```
function longestUnique(s) {
  const last = new Map();
  let l = 0, best = 0;
  for (let r = 0; r < s.length; r++) {
    if ((last.get(s[r]) ?? -1) >= l)
      l = last.get(s[r]) + 1; // jump, never move back
    last.set(s[r], r);
    best = Math.max(best, r - l + 1);
  }
  return best;
}
```

#### PROOF CUE

Explain why moving this pointer cannot discard a better answer. Sorted order or monotonic window validity must justify the move.

## 05 Stacks, monotonic structures, and binary search

### A. STACK

- Use for nesting/undo/parsing, or **unresolved elements waiting for a later answer**.
- Parentheses invariant: the stack contains unmatched openings; a closing token must match the top.
- RPN: pop operands in correct order; for subtraction/division, the **second popped is the left operand**.

### B. MONOTONIC STACK

```
const ans = new Array(a.length).fill(-1), st = [];  
for (let i = 0; i < a.length; i++) {  
  while (st.length && a[i] > a[st.at(-1)]) {  
    const j = st.pop();  
    ans[j] = i;           // i answers j's "next  
    // greater"  
  }  
  st.push(i);  
}
```

- The stack holds indices whose next-greater answer is unknown, values decreasing. The current value resolves dominated entries.
- **Histogram**: increasing stack of start-index/height; when a lower bar arrives, the popped height spans from its stored start to the current index.

### C. BINARY SEARCH: EXACT

```
let l = 0, r = a.length - 1;  
while (l <= r) {  
  const m = l + Math.floor((r - l) / 2);  
  if (a[m] === x) return m;  
  if (a[m] < x) l = m + 1; else r = m - 1;  
}  
return -1;
```

- Invariant: if the target exists, it is inside inclusive [l,r]. Every branch strictly shrinks the range.

### D. LOWER BOUND

```
let l = 0, r = a.length;    // half-open [l, r)  
while (l < r) {  
  const m = l + Math.floor((r - l) / 2);  
  if (a[m] >= x) r = m; else l = m + 1;  
}  
return l; // first index with a[i] >= x
```

- Upper bound changes the predicate to `a[m] > x`. Frequency of `x` = upper – lower.

### E. SEARCH ON ANSWER

- Phrase: **minimum X such that feasible(X)**, where feasibility is monotonic: false...false, true...true.
- Choose bounds that contain an answer. Binary search the first feasible; complexity  $O(\log \text{range} \times \text{feasibility cost})$ .

```
// e.g. Koko: min speed to finish piles in h hours  
let l = 1, r = Math.max(...piles);  
while (l < r) {  
  const m = l + Math.floor((r - l) / 2);  
  if (hoursAt(m) <= h) r = m; else l = m + 1;  
}  
return l;  
// hoursAt(s) = sum of Math.ceil(p / s) - monotone in s
```

- **Say why the predicate is monotonic.** If it can flip back, binary search is invalid.

### F. BUG CHECK

- Do not mix inclusive and half-open templates. Confirm each mid branch moves l/r **past** m where required.
- Rotated arrays: identify the sorted half, then test target membership including boundary equality.
- JS bit-shift mid coerces to signed 32-bit; use `Math.floor` for large numeric ranges.

#### INVARIANT

Binary search is not "sorted array magic." It is repeated elimination under a monotonic predicate with a precisely defined search interval.

## 06 Linked lists and pointer choreography

### A. DRAW BEFORE CODING

- Label prev, curr, next and show the **one pointer mutation**. Most bugs come from losing the remaining list.
- Use a **dummy head** when the result head may change, or deletion/insertion at the head is possible.

### B. REVERSAL

```
function reverse(head) {
  let prev = null, curr = head;
  while (curr) {
    const next = curr.next;
    curr.next = prev;
    prev = curr; curr = next;
  }
  return prev;
}
```

- Loop invariant: prev is the fully reversed prefix; curr begins the untouched suffix.

### C. FAST / SLOW

```
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

- **Middle:** when fast reaches the end, slow is the middle. Decide which middle for even length.

- **Cycle entry:** after meeting, move one pointer to head; advance both one step until equal.

### D. GAP POINTERS

- Remove nth from end: dummy → advance fast n+1 steps → move both → `slow.next` is the target.
- State the exact gap in nodes/edges to avoid off-by-one. Test single node and removing the head.

### E. MERGE / REORDER

- Merge sorted lists with dummy + tail; append the smaller node; finally append the remainder.
- Reorder list: find middle → reverse second half → alternating merge. **Cut the first half before merging** to avoid cycles.

### F. RANDOM POINTER / LRU

- Copy random list: old→new Map in two passes, or weave copies then detach. Explain the space tradeoff.
- **LRU = Map for lookup + doubly linked list for recency.** Sentinel head/tail simplify detach/insert. (In JS, `Map` preserves insertion order — re-inserting on access gives an O(1) interview-grade LRU; say you know the classic DLL version.)

#### TEST SET

Empty, one node, two nodes, even/odd length, cycle/no cycle, remove head/tail, duplicate values. Compare node identity, not value, for cycle/intersection.

## 07 Trees and tries: choose information flow

### A. TRAVERSAL CHOICE

| ORDER            | INFORMATION FLOW     | USE                               |
|------------------|----------------------|-----------------------------------|
| <b>Preorder</b>  | node before children | copy, serialize, constraints down |
| <b>Inorder</b>   | left-node-right      | BST sorted order                  |
| <b>Postorder</b> | children before node | height, diameter, subtree DP      |
| <b>BFS</b>       | level by level       | shortest edges, views, levels     |

### B. RECURSION CONTRACT

- **Say what the helper returns.** Example: height(node) returns the number of nodes on the longest downward path.
- The base case must match the units. Diameter may count edges while height counts nodes.

```
function diameter(root) {
  let best = 0;
  function h(n) {                // nodes on longest path
    if (!n) return 0;
    const l = h(n.left), r = h(n.right);
    best = Math.max(best, l + r); // edges through n
    return 1 + Math.max(l, r);
  }
  h(root); return best;
}
```

### C. BST

- Validate with **ancestor bounds**, not parent-only comparison. Decide the duplicate policy.

- Inorder is sorted; kth smallest via iterative stack can stop early. Search/insert follows ordering in O(h).

```
function valid(n, lo = -Infinity, hi = Infinity) {
  if (!n) return true;
  if (n.val <= lo || n.val >= hi) return false;
  return valid(n.left, lo, n.val)
    && valid(n.right, n.val, hi);
}
```

### D. BFS WITHOUT SHIFT

```
const q = [root]; let head = 0;
while (head < q.length) {
  const levelEnd = q.length; // capture before the level
  while (head < levelEnd) {
    const n = q[head++]; // index queue: O(1) dequeue
    if (n.left) q.push(n.left);
    if (n.right) q.push(n.right);
  }
}
```

### E. TRIE

- Use when **prefix queries over many strings** matter. Each node maps char → child plus a terminal marker.
- Insert/search/prefix O(L). Memory may dominate; alphabet arrays trade space/predictability vs Maps.
- Word Search II = grid DFS + trie prefix pruning; mark visited, restore, dedupe found words.

### F. RECURSION RISK

- Time O(nodes visited); stack O(height). A skewed tree may overflow the JS call stack; iterative traversal is safer when depth is unbounded.

## 08 Heaps, top-k, and selection

### A. RECOGNITION

- Repeatedly need the current minimum/maximum while data changes; k best; merge k sorted sources; shortest-path frontier; running median.
- A one-time sort is often simpler if all data is present and  $O(n \log n)$  is acceptable. Heap wins for streaming, or k much smaller than n.

### B. TOP-K INVERSION

- **K largest → min-heap of size k**; evict the smallest. K smallest → max-heap of size k.
- Complexity  $O(n \log k)$ , space  $O(k)$ . The final heap is not globally sorted.

### C. MINHEAP TEMPLATE (JS HAS NO BUILT-IN)

```
class MinHeap {
  constructor(cmp = (a, b) => a - b) {
    this.a = []; this.cmp = cmp;
  }
  get size() { return this.a.length; }
  push(x) {
    const a = this.a; a.push(x);
    let i = a.length - 1;
    while (i > 0) { // bubble up
      const p = (i - 1) >> 1;
      if (this.cmp(a[i], a[p]) >= 0) break;
      [a[i], a[p]] = [a[p], a[i]]; i = p;
    }
  }
  pop() {
    const a = this.a, top = a[0], last = a.pop();
    if (a.length) {
      a[0] = last;
      let i = 0; // bubble down
      while (true) {
        const l = 2*i+1, r = 2*i+2; let m = i;
        if (l < a.length && this.cmp(a[l], a[m]) < 0) m = l;
        if (r < a.length && this.cmp(a[r], a[m]) < 0) m = r;
        if (m === i) break;
        [a[i], a[m]] = [a[m], a[i]]; i = m;
      }
    }
    return top;
  }
  peek() { return this.a[0]; }
}
// max-heap: new MinHeap((a,b) => b - a)
// pairs: new MinHeap((a,b) => a[0] - b[0])
```

- Practice writing this cold — needing it and not having it burns 10 minutes. Comparator defines priority; test equal priorities and empty pop.

### D. MERGE K SORTED

- Heap holds **one frontier item per source**: [value, source, index/node]. Pop min, append, push the successor from the same source.
- $O(N \log k)$ , space  $O(k)$ , N total items. This is multiway merge, not top-k.

### E. RUNNING MEDIAN

- Max-heap low half, min-heap high half. All low  $\leq$  all high; sizes differ by at most one.
- Insert into the appropriate side, then rebalance. Median from roots.  $O(\log n)$  insert,  $O(1)$  query.

### F. QUICKSELECT ALTERNATIVE

- Partition around a pivot and recurse only into the target side: average  $O(n)$ , worst  $O(n^2)$ , mutates input.
- Prefer heap when streaming, repeated queries, or deterministic  $O(n \log k)$  is clearer. Prefer sort when simplest.

### G. BUG CHECK

- Min vs max comparator; size threshold before/after push; stable-tie requirement; overflow in subtraction comparators for extreme values.
- Dijkstra's heap may contain stale entries; skip when the popped distance is worse than best known.

#### CHOICE LINE

"All items are available and n is modest, so sorting is simplest. If this were streaming or  $k \ll n$ , I would keep a size-k heap."



## 09 Graphs: traversal, topology, and connectivity

### A. MODEL THE GRAPH

- Nodes/edges, directed or undirected, weighted or not, cycles, disconnected, implicit grid or explicit adjacency.
- Build an adjacency list  $O(V+E)$ . For undirected edges, add both directions. Decide labels and absent nodes.

### B. DFS VS BFS

| DFS                                    | BFS                             |
|----------------------------------------|---------------------------------|
| reachability, components, backtracking | fewest unweighted edges, levels |
| stack/recursion                        | queue + visited on enqueue      |
| may go deep                            | frontier memory may be wide     |

- Mark visited **when enqueueing**, not when dequeuing, to avoid duplicates. In DFS mark before recursion.

### C. GRID AS GRAPH

```
const dirs = [[1,0],[-1,0],[0,1],[0,-1]];
for (const [dr, dc] of dirs) {
  const nr = r + dr, nc = c + dc;
  if (0 <= nr && nr < R && 0 <= nc && nc < C
      && ok(nr, nc)) { ... }
}
```

- Mutate the grid for visited only if allowed; otherwise boolean matrix/Set. **Multi-source BFS** seeds every source at distance 0.

### D. TOPOLOGICAL SORT (KAHN)

- Directed prerequisites → indegrees + queue of zero-indegree nodes. Pop, append, decrement outgoing.
- **Processed count < V means a cycle.** If order must be deterministic, the queue policy may need a heap/sort.

```
const q = []; indeg.forEach((d,i) => { if (d===0) q.push(i); });
let head = 0, order = [];
while (head < q.length) {
  const u = q[head++]; order.push(u);
  for (const v of adj[u])
    if (--indeg[v] === 0) q.push(v);
}
return order.length === n ? order : [];
```

### E. CYCLE DETECTION IN A DIRECTED GRAPH (DFS COLORS)

```
const color = new Array(n).fill(0);
// 0 = white (unseen), 1 = gray (in current path), 2 = black (done)
function dfs(u) {
  color[u] = 1;
  for (const v of adj[u]) {
    if (color[v] === 1) return true; // back edge = cycle
    if (color[v] === 0 && dfs(v)) return true;
  }
  color[u] = 2;
  return false;
}
```

- Gray = on the current recursion path; hitting gray is a back edge. A plain visited set is **not enough** for directed cycles — black nodes are safe to revisit.
- Undirected cycle: DFS that sees a visited neighbor other than its parent, or union-find on edges.
- Kahn (processed < V) and DFS colors are interchangeable for course-schedule problems; know both.

### F. UNION-FIND

- Dynamic connectivity under edge additions: parent + size/rank; path compression + union by size gives near-constant amortized operations.
- Use for redundant edge, component count, Kruskal, accounts merge. **Not for paths or directed reachability.**

```
function find(x) {
  while (x !== p[x]) { p[x] = p[p[x]]; x = p[x]; }
  return x;
}
function union(a, b) {
  a = find(a); b = find(b);
  if (a === b) return false;
  if (sz[a] < sz[b]) [a, b] = [b, a];
  p[b] = a; sz[a] += sz[b];
  return true;
}
```

#### COMPLEXITY

Traversal is  $O(V+E)$ , including building/reading adjacency. A grid with  $R \times C$  cells is  $O(RC)$ , not  $O(R+C)$ .

## 10 Weighted graphs and shortest-path choices

### A. CHOOSE BY EDGE WEIGHTS

| CONDITION                         | ALGORITHM         |
|-----------------------------------|-------------------|
| Unweighted / equal weight         | BFS               |
| Non-negative weights              | Dijkstra          |
| Negative edges, no negative cycle | Bellman-Ford      |
| DAG weighted paths                | topological DP    |
| Connect all at minimum total cost | MST: Prim/Kruskal |
| All-pairs, small dense graph      | Floyd-Warshall    |

### B. DIJKSTRA INVARIANT

- $\text{dist}[v]$  = best discovered distance. The heap stores candidates [distance, node]. Pop minimum; **stale entries are skipped**.
- With non-negative weights, the first non-stale pop finalizes that node. Negative weights break this proof.

```
dist[src] = 0; pq.push([0, src]);
while (pq.size) {
  const [d, u] = pq.pop();
  if (d !== dist[u]) continue; // stale entry
  for (const [v, w] of adj[u])
    if (d + w < dist[v]) {
      dist[v] = d + w;
      pq.push([dist[v], v]);
    }
}
```

- Binary-heap complexity  $O((V+E) \log V)$ , often written  $O(E \log V)$  for connected sparse graphs.

### C. BELLMAN-FORD / LIMITED STOPS

- Relax every edge  $V-1$  rounds; a  $V$ th improvement signals a reachable negative cycle.  $O(VE)$ .

- **At most  $k$  stops:** perform  $k+1$  relaxation rounds using a **copy** of previous distances so one round adds at most one edge.

### D. MST

- **Kruskal:** sort edges, add if union-find says endpoints are separate.  $O(E \log E)$ .
- **Prim:** grow one tree via the cheapest crossing edge using a heap. Similar shape to Dijkstra but the priority is edge cost, not cumulative path distance.
- MST minimizes total connection cost, **not** the shortest route from a source.

### E. STATE-EXPANDED GRAPH

- If constraints include stops, keys, direction, or remaining eliminations, the node alone may not define state. Use **(node, resource/state)**.
- Visited must match the full state, or use a dominance rule. This is a frequent reason an apparently correct BFS/Dijkstra fails.

### F. BUG CHECK

- Directed edge orientation; unreachable result; stale heap entries; zero weights; integer range; negative edge.
- Shortest path vs minimum spanning tree vs cheapest path with an edge-count constraint are **different objectives**.

#### SAY THIS

"BFS minimizes number of edges; Dijkstra minimizes non-negative total weight; Bellman-Ford tolerates negative edges; MST minimizes total network cost."

## 11 Backtracking: enumerate without losing control

### A. DECISION TREE

- State = path + next choices + constraint summary.  
Pattern: **choose** → **recurse** → **unchoose**.
- Copy the path only when recording. Mutate one shared path during exploration for efficiency.

### B. SUBSETS / COMBINATIONS

```
function subsets(a) {
  const out = [], path = [];
  function dfs(start) {
    out.push([...path]); // copy when recording
    for (let i = start; i < a.length; i++) {
      path.push(a[i]);
      dfs(i + 1); // reuse allowed: dfs(i)
      path.pop();
    }
  }
  dfs(0); return out;
}
```

- The start index prevents different orders of the same combination.

### C. PERMUTATIONS

- At every level choose any unused index. Track a used boolean/bitmask; record when path length = n.
- Complexity  $O(n \cdot n!)$  including copying outputs. The output size is already exponential.

### D. DEDUPLICATION

- Sort input. Skip same-valued **siblings**: `if (i > start && a[i] === a[i-1]) continue;`

- For unique permutations, skip a duplicate value if the previous identical index is not used in the current branch.
- Understand tree level vs tree depth; a global Set is often slower and hides the reasoning.

### E. PRUNING

- Stop as soon as partial state cannot lead to a solution: sum exceeded for positive numbers, invalid prefix, attacked queen, missing trie prefix.
- Sort candidates to enable early **break**. Memoize by state when many paths reach the same subproblem — **that is the bridge to DP**.

### F. GRID DFS

- Mark current cell visited → explore neighbors → **restore**. Restoration is required because other branches may use the cell.
- Word search complexity can be  $O(RC \cdot \text{branching}^L)$ ; a trie prunes multiple-word search.

### G. BUG CHECK

- Pushing the path reference instead of a copy; forgotten pop/restore; wrong i vs i+1; duplicate skip at the wrong level; missing empty solution.

#### INVARIANT

path is exactly the choices made along the current root-to-node route, and all auxiliary state describes that same path. Every recursive return restores the caller's state.

## 12 Dynamic programming: make the state sentence precise

### A. THE FIVE-STEP RITUAL

- **1. State:** one sentence for dp indices/flags.
- **2. Transition:** choices and smaller states used.
- **3. Base:** smallest valid/empty states.
- **4. Order:** dependencies computed first.
- **5. Answer:** which state or aggregate is returned.

### B. TOP-DOWN TO BOTTOM-UP

- Write the recurrence first. Memoized DFS is often easiest to prove. Convert to a table when recursion depth/constant factors matter.
- The memo key must contain **every variable that changes future possibilities**; omit derived/redundant values.

### C. 1-D TAKE / SKIP

```
function rob(a) {
  let prev2 = 0, prev1 = 0;
  for (const x of a) {
    const cur = Math.max(prev1, prev2 + x);
    prev2 = prev1; prev1 = cur;
  }
  return prev1;
}
```

- State: best using the prefix through the current index. Transition: skip current, or take it plus  $i-2$ .

### D. COIN / KNAPSACK ORDER

- **Unbounded combinations:** coins outer, amount ascending. **Permutations/order-sensitive:** amount outer. **0/1:** amount descending.

- Coin change min:  $dp[0]=0$ , others Infinity;  $dp[a] = \min(dp[a], dp[a-c] + 1)$ .
- **Loop order is part of the correctness proof**, not an implementation detail.

### E. 2-D SEQUENCES / GRID

- Two indices often mean compare prefixes. **LCS:** match  $\rightarrow$  diagonal+1; else  $\max(\text{up}, \text{left})$ . **Edit distance:** match  $\rightarrow$  diagonal; else  $1 + \min(\text{insert}, \text{delete}, \text{replace}) = 1 + \min(\text{left}, \text{up}, \text{diagonal})$ .
- Grid paths: state at a cell comes from top/left. Pad the table with an extra row/column to simplify boundaries.
- **JS trap:** `Array(m).fill(Array(n).fill(0))` aliases one row. Use `Array.from({length:m}, () => Array(n).fill(0))`.

### F. RECOGNITION AND OPTIMIZATION

- Overlapping subproblems + optimal substructure. If a greedy choice lacks a proof, DP may be safer.
- Space-roll only after correctness: if row  $i$  depends solely on the previous row, retain one/two rows. Update direction matters.
- LIS:  $O(n^2)$  dp is easier;  $O(n \log n)$  tails/binary search when required.

### G. BUG CHECK

- The meaning of the dp state changes mid-solution; wrong base; answer at last index vs max anywhere; impossible-sentinel overflow; update order.

#### SAY THIS

"dp[i] means ... . From that state I can choose ... . The base is ... . I iterate in this order because each transition reads ... ."

## 13 Greedy reasoning and interval patterns

### A. GREEDY NEEDS A PROOF IDEA

- The local choice must be extendable to an optimal solution. Give an **exchange, stay-ahead, or discard** argument.
- If you cannot explain why the choice is safe, use DP/brute force rather than guessing.

### B. STANDARD GREEDY SHAPES

- **Kadane:** a negative prefix cannot help a later subarray, so restart. Track best ending here and global best.

```
let cur = a[0], best = a[0];
for (let i = 1; i < a.length; i++) {
  cur = Math.max(a[i], cur + a[i]);
  best = Math.max(best, cur);
}
```

- **Jump Game:** farthest reachable index dominates all smaller reach. Fail if the current index exceeds reach.
- **Scheduling max count:** keep the earliest finishing compatible interval, leaving the most room for the future.
- **Gas station:** if start  $i$  fails by  $j$ , no start between  $i$  and  $j$  can succeed; reset to  $j+1$  — after a total-feasibility check.

### C. INTERVAL SORT CHOICE

| GOAL                           | SORT BY                      |
|--------------------------------|------------------------------|
| Merge / insert                 | start                        |
| Max non-overlap / min removals | end                          |
| Rooms / active count           | starts and ends, or end-heap |
| Query min covering interval    | start + heap by size/end     |

### D. MERGE

```
function merge(xs) {
  if (!xs.length) return [];
  xs.sort((a, b) => a[0] - b[0]); // numeric comparator!
  const out = [xs[0].slice()];
  for (const [s, e] of xs.slice(1)) {
    const last = out.at(-1);
    if (s <= last[1]) last[1] = Math.max(last[1], e);
    else out.push([s, e]);
  }
  return out;
}
```

- Clarify closed vs half-open intervals and whether touching endpoints overlap. Avoid mutating the original interval objects unless allowed.

### E. SWEEP LINE

- Convert intervals to start/end events, sort, accumulate an active count. Tie ordering depends on endpoint semantics.
- A difference array is sweep line on bounded discrete coordinates; prefix sum reconstructs active counts.

### F. MATH / BIT QUICK HITS

- Rotate matrix clockwise = transpose + reverse rows. Spiral = four shrinking boundaries with guard checks.
- XOR cancels pairs;  $n \& (n-1)$  clears the lowest set bit. JS bitwise operations coerce to signed 32-bit.
- Fast exponentiation halves the exponent each step. Floyd detects cycles in repeated numeric transforms.

#### PROOF LINE

"Keeping the earliest finishing interval can replace any later-finishing chosen interval without reducing room for the remaining schedule."

## 14 Final pattern decision tree and debugging drill

### A. ASK IN THIS ORDER

- Is the answer contiguous? → window/prefix. Is input sorted or sortable? → pointers/binary/greedy.
- Need repeated membership/frequency? → hash. Need unresolved next-greater/nesting? → stack.
- Need k extreme repeatedly? → heap. Hierarchy/connectivity? → tree/graph traversal.
- Need all possibilities? → backtracking. Repeated subproblems/choices? → DP.
- Ordering under prerequisites? → topo. Weighted path? → choose by weight conditions (\$10.A).

### B. INVARIANT LIBRARY

| PATTERN         | INVARIANT                                      |
|-----------------|------------------------------------------------|
| Hash            | map summarizes processed prefix                |
| Window          | [l,r] is current candidate; state matches it   |
| Monotonic stack | entries wait for an answer; order maintained   |
| Binary search   | answer remains inside search interval          |
| BFS             | queue frontier is nondecreasing distance       |
| Dijkstra        | popped non-stale min is final                  |
| Backtracking    | path/aux state match current recursion route   |
| DP              | each state meaning and dependencies stay fixed |

### C. EDGE CASE MATRIX

- Empty / one element; duplicates; all same; already sorted/reverse; negative/zero; impossible answer.
- Min/max constraints; integer range; mutation allowed; disconnected/cycle; skewed tree; even/odd list.
- Closed vs half-open range; inclusive/exclusive endpoints; tie behavior; stable output ordering.

### D. JS-SPECIFIC BUGS

- Numeric sort needs a comparator — `[10,9,2].sort()` is lexicographic. `queue.shift()` is  $O(n)$ : use a head index.
- Bitwise coerces to signed 32-bit; `%` preserves the dividend's sign. 2-D `Array.fill` aliases rows.
- Map missing vs falsy: `has/??`, and `!= null` as the deliberate nullish check. Array/object keys compare by identity.
- Recursion depth may overflow. `at(-1)` runtime availability. `Number.MAX_SAFE_INTEGER` bound — `BigInt` if constraints demand.

### E. DRY-RUN TABLE

- Write variables as columns and one row per iteration — only for the tricky boundary. Include the iteration that exits.
- If output is wrong, find the **first iteration where the invariant breaks**. Fix that transition rather than patching the final answer.

### F. PRACTICE LOOP

- For each pattern: one learn problem → two variations → re-code the template next day → timed mixed problem one week later.
- Track the failure reason: recognition, invariant, implementation, edge case, complexity, or communication. Review by failure class.
- In the final month, **mixed timed sets beat category blocks** because the interview does not label the pattern.

#### FINAL FIVE-MINUTE CHECK

Contract stated; brute force named; invariant justified; code compiles mentally; normal and edge case dry-run; complexity includes space/recursion/output; no mutation or ordering assumption left implicit.